Analysis of Algorithms

Chapter - 07
Dynamic Programming

This Chapter Contains the following Topics:

1. Introduction
   i. What is Dynamic Programming?
   ii. Elements of Dynamic Programming

2. Matrix-Chain Multiplication
   i. Matrix-Chain Multiplication Problem
   ii. Algorithms
   iii. Examples

Introduction

What is Dynamic Programming?

- Dynamic programming solves optimization problems by combining solutions to subproblems.
- "Programming" refers to a tabular method with a series of choices, not "coding"
- A set of choices must be made to arrive at an optimal solution.
- As choices are made, subproblems of the same form arise frequently.
- The key is to *store* the solutions of subproblems to be *reused* in the future.
- Recall the divide-and-conquer approach:
  - Partition the problem into independent subproblems.
  - Solve the subproblems recursively.
  - Combine solutions of subproblems.
- This contrasts with the dynamic programming approach.

What is Dynamic Programming?

- Dynamic programming is applicable when subproblems are not independent.
  - i.e., subproblems share subsubproblems
  - Solve every subsubproblem only once and store the answer for use when it reappears.
- A divide-and-conquer approach will do more work than necessary.
- A dynamic programming approach consists of a sequence of 4 steps:
  - Characterize the structure of an optimal solution.
  - Recursively define the value of an optimal solution.
  - Compute the value of an optimal solution in a bottom-up fashion.
  - Construct an optimal solution from computed information

Elements of Dynamic Programming

- For dynamic programming to be applicable, an optimization problem must have:
- *Optimal substructure*
  - An optimal solution to the problem contains within it optimal solution to subproblems (but this may also mean a greedy strategy applies)
- *Overlapping subproblems*
  - The space of subproblems must be small; i.e., the same subproblems are encountered over and over

Matrix-Chain Multiplication

Matrix-Chain Multiplication

- Suppose we have a sequence or chain A1, A_2, …, A_n of *n* matrices to be multiplied.
  - That is, we want to compute the product $A_1A_2…A_n$.
- There are many possible ways (parenthesizations) to compute the product.
- Example: consider the chain A_1, A_2, A_3, A_4 of 4 matrices.
- Let us compute the product $A_1A_2A_3A_4$.
- There are 5 possible ways:
  - $(A_1(A_2(A_3A_4)))$
  - $(A_1((A_2A_3)A_4))$
  - $((A_1A_2)(A_3A_4))$
  - $((A_1(A_2A_3))A_4)$
  - $(((A_1A_2)A_3)A_4)$

Multiplication of Two Matrices

- To compute the number of scalar multiplications necessary, we must know:
- Algorithm to multiply two matrices.
- Matrix dimensions.
- **Input**: Matrices $A_{p\times q}$ and $B_{q\times r}$
- **Output**: Matrix $C_{p\times r}$ resulting from $A\cdot B$

Algorithm Matrix-Multiply($Ap\times q$, $Bq\times r$)

```
{   for i : = 1 to p do
        for j := 1 to r do
        {     C[i, j] := 0;
              for k := 1 to q do
                    C[i, j] := C[i, j] + A[i, k] · B[k, j];
        }
     return C;
}
```

- Number of scalar multiplications = pqr

Example

- Example: Consider three matrices $A_{10\times100}$, $B_{100\times5}$, and $C_{5\times50}$
- There are 2 ways to parenthesize
  - $((AB)C) = D_{10\times5} \cdot C_{5\times50}$
    - AB $\Rightarrow$ 10·100·5 = 5,000 scalar multiplications,
    - DC $\Rightarrow$ 10·5·50 = 2,500 scalar multiplications,
    - Total scalar multiplication = 7,500
  - $(A(BC)) = A_{10\times100} \cdot E_{100\times50}$
    - BC $\Rightarrow$ 100·5·50 = 25,000 scalar multiplications,
    - AE $\Rightarrow$ 10·100·50 = 50,000 scalar multiplications,
    - Total scalar multiplication = 75,000

Matrix-Chain Multiplication Problem

- Matrix-chain multiplication problem
  - Given a chain $A_1, A_2, \ldots, A_n$ of n matrices, where for $i=1, 2, \ldots, n$, matrix A_i has dimension $p_{i-1} \times p_i$.
  - Parenthesize the product $A_1 A_2 \ldots A_n$ such that the total number of scalar multiplications is minimized.
- Note that in the matrix-chain multiplication problem, we are not actually multiplying matrices.
- Our aim is only to determine the an order for multiplying matrices that has the lowest cost.
- Typically, the time invested in determining this optimal order is more than paid for by the time saved later on when actually performing the matrix multiplications, such as performing only 7,500 scalar multiplications instead of 75,000.
- Brute force method of exhaustive search takes time exponential in n.

Dynamic Programming Approach

- Step 1: The structure of an optimal solution
  - Let us use the notation $A_{i..j}$ for the matrix that results from the product $A_i A_{i+1} \ldots A_j$
  - An optimal parenthesization of the product $A_1A_2\ldots A_n$ splits the product between A_k and A_{k+1} for some integer k where $1 \le k < n$
  - First compute matrices $A_{1..k}$ and $A_{k+1..n}$; then multiply them to get the final matrix $A_{1..n}$
  - **Key observation**: parenthesizations of the subchains $A_1A_2\ldots A_k$ and $A_{k+1}A_{k+2}\ldots A_n$ must also be optimal if the parenthesization of the chain $A_1A_2\ldots A_n$ is optimal (why?)
  - That is, the optimal solution to the problem contains within it the optimal solution to subproblems

Dynamic Programming Approach (Contd.)

- Step 2: A Recursive solution
  - Let $m[i, j]$ be the minimum number of scalar multiplications necessary to compute $A_{i..j}$
  - Minimum cost to compute $A_{1..n}$ is $m[1, n]$
  - Suppose the optimal parenthesization of $A_{i..j}$ splits the product between A_k and A_{k+1} for some integer k where $i \le k < j$
  - $A_{i..j} = (A_i A_{i+1} \ldots A_k) \cdot (A_{k+1} A_{k+2} \ldots A_j) = A_{i..k} \cdot A_{k+1..j}$
  - Cost of computing $A_{i..j}$ = cost of computing $A_{i..k}$ + cost of computing $A_{k+1..j}$ + cost of multiplying $A_{i..k}$ and $A_{k+1..j}$
  - Cost of multiplying $A_{i..k}$ and $A_{k+1..j}$ is $p_{i-1} p_k p_j$
  - $m[i, j] = m[i, k] + m[k+1, j] + p_{i-1} p_k p_j$ for $i \le k < j$
  - $m[i, i] = 0$ for $i=1,2,\ldots,n$
  - But… optimal parenthesization occurs at one value of k among all possible $i \le k < j$
  - Check all these and select the best one

Dynamic Programming Approach (Contd.)

- Step 2: A Recursive solution (contd..)
  - Thus, our recursive definition for the minimum cost of parenthesizing the product $A_i A_{i+1} \ldots A_j$ becomes

$$m[i,j] = \begin{cases} 0 & \text{if } i = j, \\ \min\limits_{i \le k < j} \{m[i,k] + m[k+1,j] + p_{j-1} p_k p_j\} & \text{if } i < j. \end{cases}$$

  - To keep track of how to construct an optimal solution, let us define *s*[*i*, *j*] = value of *k* at which we can split the product $A_i A_{i+1} \ldots A_j$ to obtain an optimal parenthesization.
  - That is, s[I, j] equals a value k such that *m*[*i*, *j*] = *m*[*i*, *k*] + *m*[*k*+1, *j*] + $p_{i-1} p_k p_j$

- Step 3: Computing the optimal cost
  - Algorithm: next slide
  - First computes costs for chains of length *l*=1
  - Then for chains of length *l*=2,3, … and so on
  - Computes the optimal cost bottom-up

Dynamic Programming Approach (Contd.)

- **Input**: Array *p*[0…*n*] containing matrix dimensions and *n*
- **Result**: Minimum-cost table *m* and split table *s*

```
Algorithm MatrixChainOrder(p[ ], n)
{
    for i := 1 to n do
            m[i, i] := 0;
   for l := 2 to n do
   for i := 1 to n-l+1 do
   {    j := i+l-1;
        m[i, j] := ∞;
        for k := i to j-1 do
        {  q := m[i, k] + m[k+1, j] + p[i-1] p[k] p[j];
           if (q < m[i, j]) then
           {              m[i, j] := q;
            s[i, j] := k;
           }
   }
        return m and s
     }
```

Example

- The algorithm takes $O(n^3)$ time and requires $O(n^2)$ space.
- Example:
- Consider the following six matrix problem.

Matrix	A_1	A_2	A_3	A_4	A_5	A_6
Dimensions	10x20	20x5	5x15	15x50	50x10	10x15

- The problem therefore can be phrased as one of filling in the following table representing the values m.

i\j	1	2	3	4	5	6
1	0					
2		0				
3			0			
4				0		
5					0	
6						0

Example (Contd.)

- Chains of length 2 are easy, as there is no minimization required, so
- $m[i, i+1] = p_{i-1}p_ip_{i+1}$
- m[1, 2] = 10x20x5 = 1000
- m[2, 3] = 20x5x15 = 1500
- m[3, 4] = 5x15x50 = 3750
- m[4, 5] = 15x50x10 = 7500
- m[5, 6] = 50x10x15 = 7500

i\j	1	2	3	4	5	6
1	0	1000				
2		0	1500			
3			0	3750		
4				0	7500	
5					0	7500
6						0

Example (Contd.)

- Chains of length 3 require some minimization – but only one each.
- $m[1,3]=\min\{(m[1,1]+m[2,3]+p_0p_1p_3),(m[1,2]+m[3,3]+p_0p_2p_3)\}$
  $= \min\{(0+1500+10x20x15), (1000+0+10x5x15)\}$
  $= \min\{ 4500, 1750 \} = 1750$
- $m[2,4]=\min\{(m[2,2]+m[3,4]+p_1p_2p_4),(m[2,3]+m[4,4]+p_1p_3p_4)\}$
  $= \min\{(0+3750+20x5x50), (1500+0+20x15x50)\}$
  $= \min\{ 8750, 16500 \} = 8750$
- $m[3,5]=\min\{(m[3,3]+m[4,5]+p_2p_3p_5),(m[3,4]+m[5,5]+p_2p_4p_5)\}$
  $= \min\{(0+7500+5x15x10), (3750+0+5x50x10)\}$
  $= \min\{ 8250, 6250 \} = 6250$
- $m[4,6]=\min\{(m[4,4]+m[5,6]+p_3p_4p_6),(m[4,5]+m[6,6]+p_3p_5p_6)\}$
  $= \min\{(0+7500+15x50x15), (7500+0+15x10x15)\}$
  $= \min\{ 18750, 9750 \} = 9750$

i\j	1	2	3	4	5	6
1	0	1000	1750			
2		0	1500	8750		
3			0	3750	6250	
4				0	7500	9750
5					0	7500
6						0

Example (Contd.)

- $m[1,4]=\min\{(m[1,1]+m[2,4]+p_0p_1p_4),(m[1,2]+m[3,4]+p_0p_2p_4),$
  $(m[1,3]+m[4,4]+p_0p_3p_4)\}$
  $= \min\{(0+8750+10x20x50), (1000+3750+10x5x50),$
  $(1750+0+10x15x50)\}$
  $= \min\{ 18750, 7250, 9250 \} = 7250$
- $m[2,5]=\min\{(m[2,2]+m[3,5]+p_1p_2p_5),(m[2,3]+m[4,5]+p_1p_3p_5),$
  $(m[2,4]+m[5,5]+p_1p_4p_5)\}$
  $= \min\{(0+6250+20x5x10), (1500+7500+20x15x10),$
  $(8750+0+20x50x10)\}$
  $= \min\{ 7250, 12000, 18750 \} = 7250$
- $m[3,6]=\min\{(m[3,3]+m[4,6]+p_2p_3p_6),(m[3,4]+m[5,6]+p_2p_4p_6),$
  $(m[3,5]+m[6,6]+p_2p_5p_6)\}$
  $= \min\{(0+9750+5x15x15), (3750+7500+5x50x15),$
  $(6250+0+5x10x15)\}$
  $= \min\{ 10875, 15000, 7000 \} = 7000$

i\j	1	2	3	4	5	6
1	0	1000	1750	7250		
2		0	1500	8750	7250	
3			0	3750	6250	7000
4				0	7500	9750
5					0	7500
6						0

Example (Contd.)

- $m[1,5]=\min\{(m[1,1]+m[2,5]+p_0p_1p_5),(m[1,2]+m[3,5]+p_0p_2p_5),$
  $(m[1,3]+m[4,5]+p_0p_3p_5),(m[1,4]+m[5,5]+p_0p_4p_5)\}$
  $= \min\{(0+7250+10x20x10), (1000+6250+10x5x10),$
  $(1750+7500+10x15x10), (7250+0+10x50x10)\}$
  $= \min \{ 9250, 7750, 10750, 12250 \} = 7750$
- $m[2,6]=\min\{(m[2,2]+m[3,6]+p_1p_2p_6),(m[2,3]+m[4,6]+p_1p_3p_6),$
  $(m[2,4]+m[5,6]+p_1p_4p_6),(m[2,5]+m[6,6]+p_1p_5p_6)\}$
  $= \min\{(0+7000+20x5x15), (1500+9750+20x15x15),$
  $(8750+7500+20x50x15), (7250+0+20x10x15)\}$
  $= \min \{ 8500, 15750, 31,250, 10250 \} = 8500$
- $m[1,6]=\min\{(m[1,1]+m[2,6]+p_0p_1p_6),(m[1,2]+m[3,6]+p_0p_2p_6),$
  $(m[1,3]+m[4,6]+p_0p_3p_6),(m[1,4]+m[5,6]+p_0p_4p_6),$
  $(m[1,5]+m[6,6]+p_0p_5p_6)\}$
  $= \min\{(11500, 8750, 13750, 22250, 9250 \} = 8750$

i\j	1	2	3	4	5	6
1	0	1000	1750	7250	7750	8750
2		0	1500	8750	7250	8500
3			0	3750	6250	7000
4				0	7500	9750
5					0	7500
6						0

Dynamic Programming Approach (Contd.)

- The algorithm takes $O(n^3)$ time and requires $O(n^2)$ space.
- Step 4: Constructing an optimal solution
  - Our algorithm computes the minimum-cost table *m* and the split table *s.*
  - The optimal solution can be constructed from the split table *s.*
  - Each entry $s[i, j]=k$ shows where to split the product $A_i A_{i+1} \ldots A_j$ for the minimum cost.
  - The following recursive procedure prints an optimal parenthesization.

```
Algorithm PrintOptimalPerens(s, i, j)
{     if (i=j) then
            Print "A"i;
      else
      {     Print "(";
            PrintOptimalPerens(s, i, s[i,j]);
            PrintOptimalPerens(s, s[i,j]+1, j);
            Print ")";
      }
}
```

Example (Contd.)

- So far we have decided that the best way to parenthesize the expression results in 8750 multiplication.
- But we have not addressed how we should actually DO the multiplication to achieve the value.
- However, look at the last computation we did – the minimum value came from computing

$$A = (A_1A_2)(A_3A_4A_5A_6)$$

- Therefore in an auxiliary array, we store value s[1,6]=2.
- In general, as we proceed with the algorithm, if we find that the best way to compute $A_{i..j}$ is as

$$A_{i..j} = A_{i..k}A_{(k+1)..j}$$

then we set

$$s[i, j] = k.$$

- Then from the values of k we can reconstruct the optimal way to parenthesize the expression.

Example (Contd.)

- If we do this then we find that the s array looks like this:

i\j	1	2	3	4	5	6
1	1	1	2	2	2	2
2		2	2	2	2	2
3			3	3	4	5
4				4	4	5
5					5	5
6						6

- We already know that we must compute $A_{1..2}$ and $A_{3..6}$.
- By looking at s[3,6] = 5, we discover that we should compute $A_{3..6}$ as $A_{3..5}A_{6..6}$ and then by seeing that s[3,5] = 4, we get the final parenthesization

$$A = ((A_1A_2)(((A_3A_4)A_5)A_6)).$$

- And quick check reveals that this indeed takes the required 8750 multiplications.